Game ability virtual console

Chapter 3 · Practical Lab 2 · Virtual functions and abstract interfaces

In this practical, you will make two simple game abilities respond through the same class interface. You will first see what happens with an ordinary non-virtual function, then make the call virtual, and finally turn the base class into an abstract interface.

You will write the C++ definitions and most of the console statements yourself. Class declarations, small hints, required behaviours and expected outputs are provided. The challenge is to connect those pieces, not to copy a finished program.

Duration: 120 minutes · C++17 · Visual Studio 2026 · Start from an Empty Project

> How to work. Read one section, predict what should happen, add only the requested code, build, and compare with the expected output. If it differs, fix the current stage before moving on. You do not need to invent a test framework.

Before you begin

You need Visual Studio with the Desktop development with C++ workload. Use Platform Toolset v145, ISO C++17, /W4, Debug and x64. The previous practical introduced public inheritance, constructors, const member functions, references and pointers; this lab builds directly on those skills.

What you will practise

- distinguishing a non-virtual call from a virtual call through a base reference;
- using override to make the compiler check a matching member-function signature;
- recognising a pure virtual function and an abstract class;
- releasing one owned derived object safely through a virtual base destructor;
- using a single caller to work with different concrete ability types; and
- checking small input rules before accessing a pointer array.

Not required

No game engine, graphics, inheritance hierarchies beyond one base and two derived classes, dynamic_cast, RTTI, vtable inspection, templates, containers, exception-handling exercise or separate automated tests. The goal is the behaviour of one shared interface.

The problem

A game has different abilities. A shield raises protection, while a healing ability restores points. They both have a power value and can be activated, but they must display different actions. A simple game system should not need separate code paths for every type of ability.

You will use GameAbility as their common base and ShieldAbility and HealAbility as the concrete classes. The power value is intentionally small (0–30). We will treat power 0 as an inactive ability when the runner is added. All supplied objects obey the range; constructor validation is not part of the core lab.

Files you will create

File	Purpose
Main.cpp	Create concrete objects, provide borrowed base views and print labelled results.
Abilities.h	Declare the common base and both concrete classes.
Abilities.cpp	Write constructors, activation functions, queries and destructor traces.
Runner.h	Declare one caller using a forward-declared GameAbility pointer type.
Runner.cpp	Implement the loop and small validity checks through the common interface.

Your two-hour route

Stage	Focus	Time
01	Create the Visual Studio project from scratch	12 min
02	Write a base and two concrete abilities	25 min
03	Turn non-virtual behaviour into virtual dispatch	20 min
04	Make the base operation pure virtual	15 min
05	Show safe deletion through a virtual base destructor	18 min
06	Build a shared ability runner	22 min
07	Complete the output and short evidence record	8 min

01 · Create the project from scratch

In Visual Studio, choose Create a new project → Empty Project (C++). Name it **GameAbilityVirtualLab**. If asked, use the same name for the solution. This should begin as a genuinely empty C++ project rather than an imported starter project.

Select Debug and x64. Under Project → Properties (All Configurations), set Platform Toolset to v145, C++ Language Standard to ISO C++17, Warning Level to Level 4 (/W4), and make sure the output is a Console application. Use the usual C++ exception option /EHsc and disable precompiled headers.

Under Source Files, add Main.cpp. Type this starter:

```
#include <iostream>

int main()
{
    std::cout << "Game ability lab\n";
    return 0;
}
```

Build (Ctrl+Shift+B) and run without debugging (Ctrl+F5). You should see one line:

```
Game ability lab
```

Create Abilities.h and Abilities.cpp now. Later, in Stage 06, you will add Runner.h and Runner.cpp. Keep one main() in the whole solution.

> Checkpoint 01. Confirm that the project builds and prints its heading before adding any inheritance. Resolve project configuration problems here rather than while writing class code.

02 · Declare the common GameAbility class

GameAbility is our general ability. It keeps its power private, offers a read-only power() query, and declares activate(). For now, activate() is deliberately NOT virtual, so you can compare the result later.

In Abilities.h, enter this complete starting interface. Do not change void activate() const to virtual yet.

```
#ifndef ABILITIES_H
#define ABILITIES_H

class GameAbility
{
public:
    explicit GameAbility(int t_power);
    virtual ~GameAbility() = default;
    void activate() const;           // Not virtual yet.
    int power() const;

private:
    int m_power;
};

class ShieldAbility : public GameAbility
{
public:
    explicit ShieldAbility(int t_power);
    void activate() const;
};

class HealAbility : public GameAbility
{
public:
    explicit HealAbility(int t_power);
    void activate() const;
};

#endif
```

Both derived classes inherit GameAbility's power value; neither needs a second m_power. Each derived class will have a function named activate(), but those functions are not overriding a virtual operation yet.

Write the definitions in Abilities.cpp

Include Abilities.h first, followed by <iostream>. Write the definitions in the following order. Use GameAbility::, ShieldAbility:: and HealAbility:: correctly in the function headers. You are responsible for the function bodies.

Definition	What your code must do
GameAbility(int)	Initialise m_power from t_power using a member initialiser list.
GameAbility::power() const	Return the stored power without changing it.
GameAbility::activate() const	Print Generic ability followed by a newline.
ShieldAbility(int)	Initialise the direct GameAbility base with t_power.
ShieldAbility::activate() const	Print Shield power: followed by the inherited power().
HealAbility(int)	Initialise the direct GameAbility base with t_power.
HealAbility::activate() const	Print Healing points: followed by power().

Hints. A derived constructor initialises its base with : GameAbility{t_power}. A derived activation function can call power() because it is public. The stored m_power is private to GameAbility: do not try to read it directly from ShieldAbility or HealAbility.

Check the non-virtual result

In Main.cpp, include Abilities.h. After the heading, create a short inner block that contains these declarations. Add three small output labels and call activate() through each route. Do not write a whole new main() each time.

```
ShieldAbility shield{20};
HealAbility heal{15};
GameAbility const& view{shield};

// "Direct: "       followed by shield.activate().
// "Base view: "    followed by view.activate().
// "Second type: "  followed by heal.activate().
```

Add the section title Non-virtual check above the calls. Before running, predict whether the base view uses the Shield or GameAbility body. Then compare:

```
Non-virtual check
Direct: Shield power: 20
Base view: Generic ability
Second type: Healing points: 15
```

> Checkpoint 02. A reference still refers to the complete ShieldAbility. It prints Generic ability because the base activate() operation is currently non-virtual, not because the shield has been copied.

03 · Make the calls virtual

You have already observed the problem. Keep the same class relationship and the same main() calls, but change the GameAbility::activate declaration in Abilities.h to virtual. Then add override to the matching declaration in each derived class.

```
// Replace only these declarations in Abilities.h:
virtual void activate() const;

// In each derived class:
void activate() const override;
```

Do not create a second GameAbility class or a second definition of the same function. Keep the existing function bodies in Abilities.cpp unchanged: their signatures already match.

Run the same three calls. Rename the earlier heading to Virtual calls. The new result should be:

```
Virtual calls
Direct: Shield power: 20
Base view: Shield power: 20
Second type: Healing points: 15
```

This time, the call through GameAbility const& selects the derived function because activate is virtual. The reference still does not create or own another object.

> One safe build-only experiment. In ShieldAbility’s declaration, temporarily remove const but leave override. Build and inspect the first relevant compiler diagnostic. Restore const and rebuild. Do not remove override just to hide the mistake.

> Checkpoint 03. The virtual call uses the same main() statement as before; only the function declarations changed. You can explain why override detects the incorrect signature.

04 · Turn GameAbility into an abstract class

A generic ability cannot sensibly choose whether to raise a shield or apply healing. Instead of keeping the placeholder Generic ability implementation, require each concrete class to supply its own activation behaviour.

Replace the base declaration with this pure virtual form:

```
virtual void activate() const = 0;
```

Remove only the old GameAbility::activate() definition from Abilities.cpp. Keep the base constructor, destructor and power query. Leave both concrete activate definitions in place.

The = 0 is a pure specifier: it does not tell the function to return zero. It makes GameAbility abstract, so code cannot create a complete GameAbility object. The ShieldAbility and HealAbility objects are still valid because they fulfil the requirement.

Check this without running invalid code: temporarily add GameAbility generic{10}; to your main block. Build, read why instantiation fails, then remove the line. Run the restored project; the earlier Virtual calls section should produce exactly the same three lines as Stage 03.

> Think about the design. A derived class with void activate() (missing const) would not fulfil activate() const. Keeping override lets the compiler report the mismatch at the declaration instead of quietly leaving a required operation unimplemented.

> Checkpoint 04. GameAbility is abstract, but its constructor and private power member still form the base part of every complete ShieldAbility and HealAbility object.

05 · Release an owned ability safely

The earlier practical introduced new and delete. We will use just one owned object here, so the focus remains on virtual destruction rather than memory-management complexity.

In Abilities.h, replace the defaulted base destructor declaration with virtual ~GameAbility();. Add ~HealAbility() override; inside HealAbility's public section. Do not add extra destructor declarations to ShieldAbility.

In Abilities.cpp, define these destructors. The base destructor should print - GameAbility; the HealAbility destructor should print - HealAbility. The names must match exactly, including the ~ symbol.

Destructor	Required output
GameAbility::~GameAbility()	- GameAbility
HealAbility::~HealAbility()	- HealAbility

Inside the existing main() scope, after the virtual calls, add a new labelled section called Owned action. Construct one HealAbility dynamically, but store its address in a GameAbility* named pOwned. Activate the object through that pointer, then use one matching delete. Finally set pOwned to nullptr.

```
// Start of the owned section:
GameAbility* pOwned{new HealAbility{7}};
// Call the inherited virtual interface.
// Delete this ONE owned allocation exactly once.
// Set pOwned to nullptr afterwards.
```

Expected owned-action section

```
Owned action
Healing points: 7
- HealAbility
- GameAbility
```

The derived destructor runs first, followed by its base destructor. A virtual base destructor makes this specific deletion through GameAbility* valid. Do not delete pointers to your earlier local shield or heal objects; those pointers would only be borrowed views.

> Important distinction. Do not temporarily remove virtual and run delete pOwned to "see the result". Deleting a derived object through a non-virtual public base destructor can cause undefined behaviour. The correct version is the only one to execute.

> Checkpoint 05. The owned action prints two destructor messages. Your other local objects will also be destroyed when their scope ends; in the final run those messages appear after Leaving scope.

06 · Write a runner that works with any ability

A useful game system should not need an if statement for ShieldAbility and another for HealAbility. It should ask the common GameAbility interface to activate whichever concrete object occupies a valid slot.

Create Runner.h and Runner.cpp now. Type the following declaration in Runner.h. The header only needs the class name, because the parameter contains pointers rather than GameAbility values.

```
#ifndef RUNNER_H
#define RUNNER_H

class GameAbility;

int activateReady(GameAbility const* const* t_abilities,
                  int t_count);

#endif
```

In Runner.cpp, include Runner.h first and Abilities.h second. Write the whole definition of activateReady() yourself using the following rules. No type-specific branches or dynamic allocations are needed.

Input / situation	Required behaviour
Count less than 0 or more than 4	Return -1 before accessing any slots.
Count equals 0	Return 0, even when t_abilities is nullptr.
Positive count; array pointer is nullptr	Return -1 before indexing.
A slot is nullptr	Skip that slot.
An ability has power 0	Skip it: this ability is inactive.
A live ability has positive power	Call activate() once and count it.
After examining all slots	Return the number of activations; do not change the inputs.

> Suggested logic order: validate the count → handle an empty array → reject missing storage → start an integer counter at 0 → loop from 0 to count - 1 → check the pointer before calling power() → activate only a positive-power object → return the counter. You have already used if statements, for loops and -> in earlier labs.

If a positive count is supplied, a non-null array pointer must refer to at least that many real slots. Every non-null slot must refer to a live object. The runner can check null pointers, but cannot prove that a dangling pointer is safe.

Call your runner

In Main.cpp, include Runner.h. Reuse the existing local shield and heal. Add these three pointer slots and call your function:

```
GameAbility const* abilities[]{&shield, nullptr, &heal};
int activated{activateReady(abilities, 3)};

// Print "Activated: " followed by activated.
// Print "Powers: " followed by shield.power() and heal.power().
```

The function should print only two activation messages, because the middle slot is null. The stored power values should not change:

```
Ability runner
Shield power: 20
Healing points: 15
Activated: 2
Powers: 20 15
```

Check two boundaries

You do not need to invent tests. Under the title Boundary checks, print the returned values from activateReady(nullptr, 0) and activateReady(nullptr, 1). The expected result is:

```
Boundary checks
Empty: 0
Missing array: -1
```

Why the order matters. If count is zero, the function returns 0 without looking at storage. If count is positive but the array pointer is null, it returns -1. Neither case should dereference nullptr.

Checkpoint 06. The runner can handle ShieldAbility and HealAbility without naming either concrete class in Runner.cpp. All code still builds with C++17, /W4 and the originally supplied power values.

07 · Put Main.cpp into its final form

You now have the required class code. Keep the includes and calls you have already written. Arrange them in one inner scope, in the order below, so the last two local objects are destroyed together at the end. You are responsible for the few remaining cout labels; use the final expected output rather than copying another large driver program.

```
// Inside main(), after the heading:
{
    ShieldAbility shield{20};
    HealAbility heal{15};
    GameAbility const& view{shield};

    // Virtual calls: direct shield, base view, direct heal.

    GameAbility const* abilities[]{&shield, nullptr, &heal};
    int activated{activateReady(abilities, 3)};
    // Runner results and unchanged power values.

    // Empty and missing-array calls to activateReady.

    GameAbility* pOwned{new HealAbility{7}};
    // Activate, delete once, then assign nullptr.

    // Print "Leaving scope" before the closing brace.
}
```

Only after the core program is correct, remove duplicated statements left from earlier experiments. Keep exactly one main() and use matching declarations and definitions in the correct files.

Final expected console output

```
Game ability lab

Virtual calls
Direct: Shield power: 20
Base view: Shield power: 20
Second type: Healing points: 15

Ability runner
Shield power: 20
Healing points: 15
Activated: 2
Powers: 20 15

Boundary checks
Empty: 0
Missing array: -1

Owned action
Healing points: 7
- HealAbility
- GameAbility

Leaving scope
- HealAbility
- GameAbility
- GameAbility
```

The final three destructor lines belong to the two local concrete objects. heal is destroyed first because it was constructed after shield. The implicitly generated ShieldAbility destructor has no print statement, but its GameAbility base destructor still runs.

> Final checkpoint. Compare your console text with the output above, confirm there are no build errors, and save the evidence template. A correct result means the code works along the demonstrated path; the explanations show that you understand why.

If something is not working

Symptom	Start by checking
Unresolved external symbol	Is the member declaration matched by a definition in Abilities.cpp, and is that .cpp file part of the project?
Override does not override	Does const appear after the parameter list on both the base and derived declarations?
Cannot create abstract class	Does each concrete class implement activate() const? Are you accidentally creating GameAbility itself?
Base view prints Generic ability	Did you make activate() virtual in the base and rebuild the updated code?
Runner returns -1 unexpectedly	Is the count valid and, for a positive count, is the array pointer non-null?
Wrong number of activations	Are you skipping null slots and abilities with power 0, and incrementing only after activation?
Destructor output differs	Check the lifetime of the dynamic object, then the reverse order of local destruction.

Quick understanding check

- Why does a GameAbility const& call reach ShieldAbility::activate() after the base operation becomes virtual?
- Why does making activate() pure prevent a complete GameAbility object without preventing concrete abilities?
- Why does activateReady() work without including a ShieldAbility-specific branch?
- Why is deleting pOwned allowed, while deleting a pointer that merely refers to local shield would be wrong?

Optional stretch · Add a third ability

Only attempt this if the core lab is complete. Add DashAbility as another public GameAbility derived class. Give it a constructor that forwards power to the base and implement void activate() const override to print Dash power: followed by its own power. Then insert one local DashAbility into your existing borrowed pointer array and confirm that activateReady works without changes to Runner.cpp.

If you want one further recognition check, declare class DashAbility final. This prevents later classes from inheriting from DashAbility, but does not change virtual activation. You are not required to implement a fourth class.